KR0465:

The Adaptive Quest Engine - Gamified Personalized Learning

Target Profile: EdTech Innovators, Full-Stack Developers, Frontend Wizards

The Scenario

Standard, linear curriculums leave struggling students behind and bore advanced students. Campus Cortex AI already uses AI to analyze student performance and identify strengths and weaknesses.

Now, we want to take those insights and turn the standard syllabus into a personalized video game. If a student failed "Algebra" last week but aced "Geometry," their dashboard should adapt automatically, offering an "Algebra Remedial Quest" before letting them unlock the next chapter.

The Objective

Build a dynamic web-based Student Dashboard that ingests a student's past performance data and dynamically generates a gamified "Learning Path" (or Skill Tree) for a specific subject.

Phase 1: The Database & Mock Data Engine

We do not provide a dataset. Your first technical challenge is generating structured data that mirrors our production schema.

1. **Schema Implementation:** Set up a database using the following core Campus Cortex AI structures:
    - user (Role: student).
    - assignment_submission (containing score, total, percentage, and the details_json breakdown of which questions they got right/wrong).
    - questions (The MCQ bank containing subject, chapter, difficulty, question_text, and options).
2. **The Curriculum Tree:** Create a custom table/collection that maps out a standard subject (e.g., Grade 10 Math), including chapters, sub-topics, and prerequisites (e.g., You must complete *Addition* before *Multiplication*).

3. **Data Seeding:** Write a script to programmatically seed **three distinct student profiles**:
    - *The Struggling Student* (low scores in basic assignments).
    - *The Average Student*.
    - *The Advanced Student* (high scores across the board).

Phase 2: The Recommendation Engine (The Brain)

Write an algorithm or utilize an LLM to analyze a selected student's assignment_submission history against the Curriculum Tree.

Your engine must output an analysis matching the Campus Cortex AI production format:

- "strengths": What chapters the student has mastered.
- "weaknesses": Where the student is failing.
- "recommendation": The exact *next best topic/quest* they need to study.

Phase 3: The Gamified UI (The "Quest Map")

Build a visually engaging frontend. Do not build a boring list of chapters.

1. **The Skill Tree:** Build a "Quest Map" or interactive timeline. It should feature gamified elements like Experience Points (XP), locked/unlocked nodes, and visual progress bars.
2. **Dynamic Rendering:** The UI must be completely dynamic. Provide a dropdown to switch between the 3 mock student profiles. When you switch to "The Advanced Student," the UI must instantly change to show basic quests as "Completed" and unlock advanced modules.

Phase 4: The Learning Node (Micro-Lesson)

When a user clicks on their active "Recommended Quest", open a modal or new page simulating a micro-lesson.

- Fetch a relevant question from your questions table.
- The student must answer the MCQ correctly to "clear the quest" and earn XP.

The "Hired on the Spot" Differentiator (Bonus)

- **Real-time AI Adjustment:** If the student clicks into a "Quest" and gets the simulated quiz question *wrong*, the UI dynamically updates to insert a new "Refresher Quest" into their path right before their eyes, demonstrating true real-time adaptive learning.

Note on APIs: You are expected to use free-tier API keys (like Google Gemini Free Tier, Supabase Free Tier, or Twilio Sandbox)